Islamic University of Technology

Department of Computer Science and Engineering (CSE)

B.Sc. in Software Engineering

SWE 4304: Software Project Lab I

Winter, 2025-2026

Final Project Report

The 'Drim' Programming Language

Group 7

Md. Muntahi Hasan Akhiar, 230042118

S.M. Tahsinuzzaman Emon, 230042104

Tamim Mahrush Naser, 230042133

Supervisor

Farzana Tabassum,

Lecturer, CSE

Date: March 23, 2026

Contents

1 Problem Statement **2**

2 Potential Solution **2**

3 Implemented Features **3**

4 System Architecture Flow **4**

5 Class Diagram **6**

6 Design Principles **7**

7 Tools and Technology **7**

8 Application Domain **8**

9 Project Timeline **8**

10 Contribution **9**

11 Future Work **9**

12 Source Repository **9**

1 Problem Statement

Standard programming languages can be hard for beginners because they require manual memory management and strict variable declarations. This adds a lot of extra work when someone just wants to write and run code quickly, specially beginners.

Another problem is that most languages do not have built-in functions for things like physics formulas or unit conversions, forcing users to find and import external libraries.

Beyond these technical issues, we also wanted to address a personal motivation. The name **"Drim"** started as an inside joke and a meme in our student batch from our first day at university. We wanted to build something that would keep this memory alive through custom keywords like `drim` and `drimming`.

From an engineering perspective, we also wanted to master language design and memory management by building every component manually.

2 Potential Solution

To solve these problems, a new interpreted language called "Drim" was created. It is designed to be simple and easy to use for beginners. **Drim** is a custom-built, lightweight language that runs as a Command Line Interface (CLI) tool. It uses a **Tree-Walk Interpreter**, which means it reads `.drim` files, understands the code, and runs it directly in real-time. Everything in Drim was built from scratch using **C++ 17** to ensure full control over the language pipeline.

In Drim, variables are created automatically when they are first used or assigned a value, and their types are handled by the language. The language includes built-in functions for physics calculations and unit conversions so that users don't have to worry about external libraries. To keep the code safe and reliable, the interpreter uses a system where function arguments are kept strictly inside their own local space. This ensures that even during recursion, variables do not interfere with each other.

3 Implemented Features

The project includes a complete system for reading, parsing, and running code. The flow of the language follows a standard interpreter pipeline, including a Lexer (Tokenizer), a Parser that builds an Abstract Syntax Tree (AST), and an Interpreter to execute the logic.

The following features were implemented:

- **Input/Output:** Accepts user input through the `drim()` (input) command and shows output using `wake()` (output).
- **Automatic Variable Assignment:** If a value is assigned to a variable that hasn't been created yet, the system automatically creates it with the correct type.
- **Dynamic Type Casting:** Variables can hold different types like Strings, Integers, or Booleans, and the system handles the conversion automatically.
- **Data Structures Library:** Built-in support for fundamental data structures, including **Stacks** and **Queues**, for advanced data management.
- **High-Precision Calculations:** Implementation of **int64** support for larger integer calculations and increased internal precision.
- **Mathematical Operations:** Full support for arithmetic including addition (+), subtraction (-), multiplication (*), division (/), modulo (%), and power ($\wedge$).
- **Bitwise Operations:** Supports bitwise AND (&), OR (|), NOT ($\sim$), and bit-shifting («, ») for low-level data manipulation.
- **Comparison & Logical Operators:** Full set of comparison operators ($<, >, <=, >=, ==, !=$) and logical operators (`and`, `or`) for complex decision making.
- **Built-in Physics Functions:** Direct support for formulas like mass-energy ($E = mc^2$), force ($F = ma$), and more.
- **Unit Conversion Engine:** Built-in tools to convert between things like inches and centimeters, Fahrenheit and Celsius, or kilometers and feet.
- **Control Flow:** Supports conditional logic (if-else), loops (`drimming`), and loop control using `stopdrim` (break) and `drimagain` (continue).
- **Custom Syntax:** Uses unique keywords like `drimming` for loops and `wake` for output to match the project's theme.
- **String Interpolation:** Allows variables to be put directly inside strings (like `"Value: {x}"`) for easier printing.
- **Error Reporting:** Provides clear error messages with exact locations when something goes wrong in the code.
- **Commenting System:** Supports single-line comments using `//`.
- **Headless Execution:** Runs as a lightweight tool in the terminal without needing a complex interface.

- **Multi-Assignment:** Allows assigning values to multiple variables at once using commas.
- **Output Formatting:** Provides options for formatted output, including support for newlines (`wakef()` vs `wake()`) within strings.
- **Arrays:** Supports array data structures for storing lists of homogeneous values and performing operations on them.
- **Type Checking:** Built-in support for identifying variable types at runtime using the `type()` keyword.
- **Scoped Variables:** Ensures variables stay within the block where they were created.
- **User-Defined Functions:** Users can create their own functions that run during the program.
- **Recursion Support:** The language can handle functions that call themselves, allowing for more complex programming patterns.

4 System Architecture Flow

Drim's architecture is based on a classic Tree-Walk Interpreter design. The process kicks off with the source code (`.drim` file) being read as raw text and passed to the **Lexer**. The lexer scans through the characters to group them into **Tokens** like keywords, identifiers, and operators. These tokens then move on to the **Parser**, which follows the language's grammar to build an **Abstract Syntax Tree (AST)**. This tree serves as a structural map of the entire program's logic. Finally, the **Interpreter** walks through the AST node by node, executing commands while managing variables and functions through a **Scope** system. It also hooks into specialized modules like the Physics and Data Structures engines whenever the code calls for them.

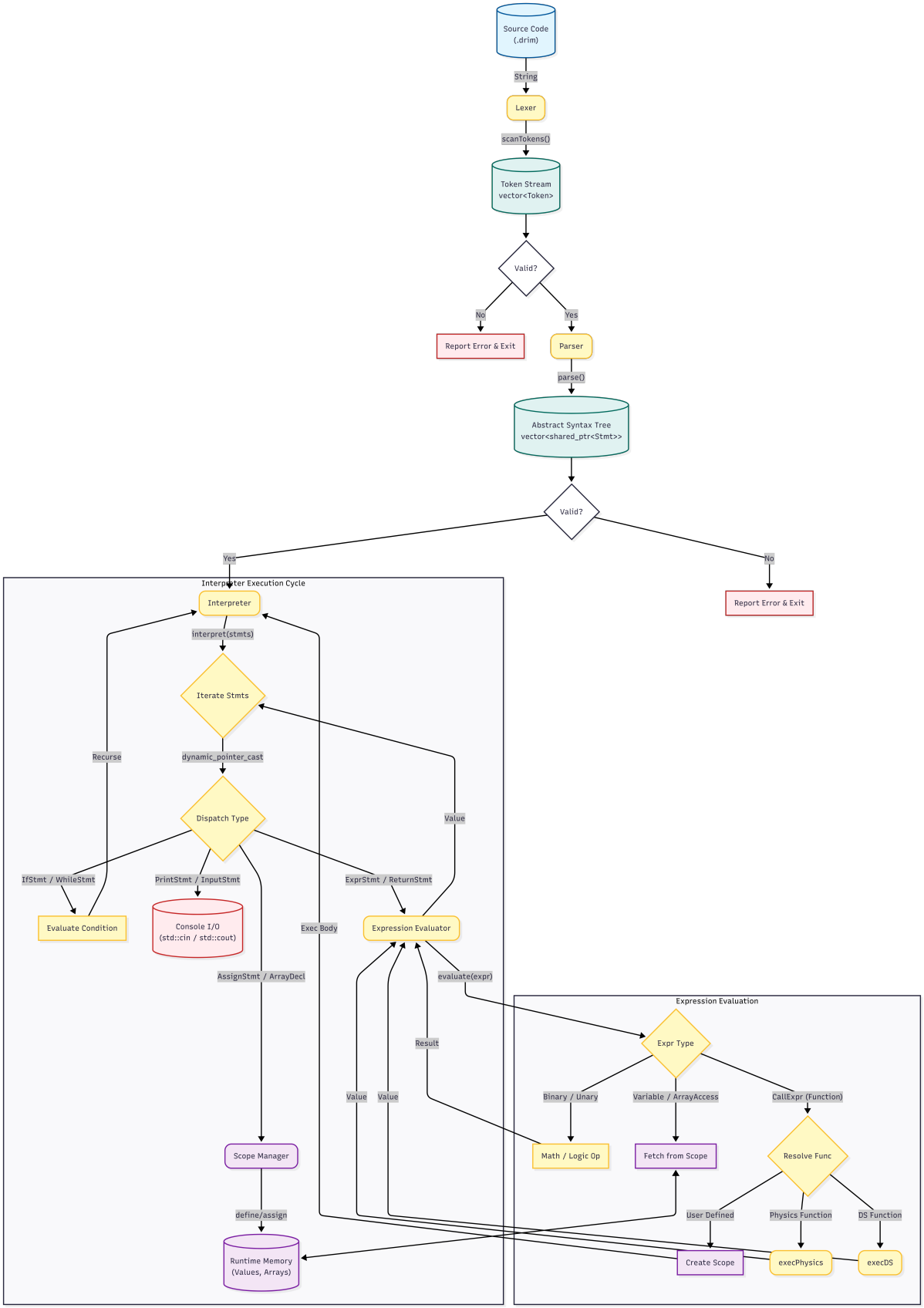

Figure 1: System Architecture Flow: From Source Code to Execution

5 Class Diagram

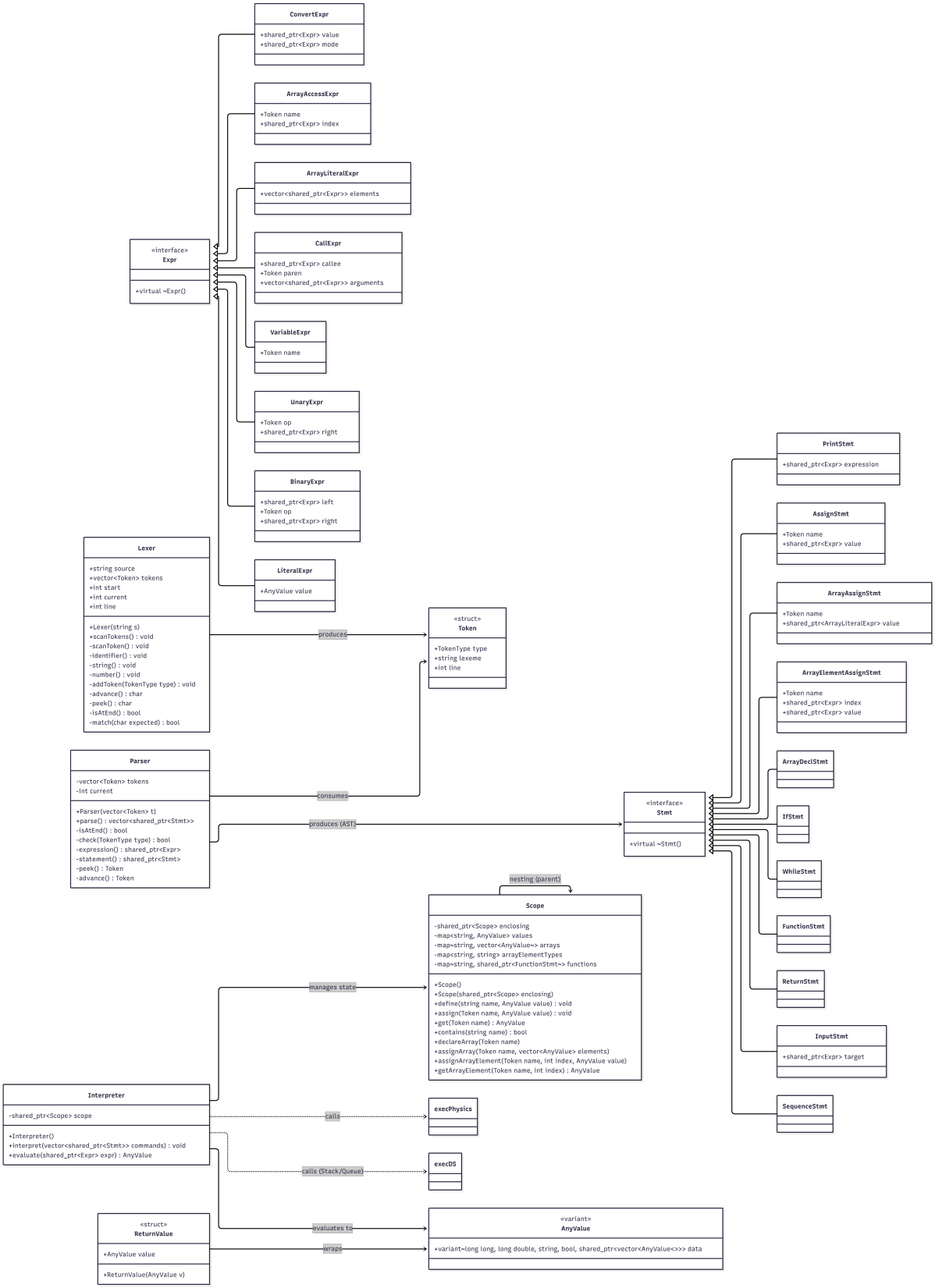

Figure 2: Project Class Diagram

The class diagram shows how Drim is structured under the hood using object-oriented principles. The language's syntax revolves around two main base structures: `Expr` (Expressions) for things that result in values, and `Stmt` (Statements) for actions. Various specific nodes, such as `BinaryExpr` for math or `IfStmt` for control flow, inherit from these bases. The `Interpreter` acts as the central engine that evaluates these nodes, relying heavily on the `Scope` class to handle memory, nested blocks, and user-defined functions. Other utility modules like the `Lexer`, `Parser`, and specialized libraries (`Physics`, `DS`) integrate into this core structure, keeping the architecture modular and decoupled.

6 Design Principles

The development of Drim was guided by the **SOLID** principles of object-oriented design to ensure the codebase remains maintainable and scalable:

- **Single Responsibility Principle (SRP):** Each core component is dedicated to a specific task. The `Lexer` is responsible solely for tokenization, and the `Interpreter` for logic execution. This separation ensures that changes to the grammar do not affect the execution engine.
- **Liskov Substitution Principle (LSP):** The Abstract Syntax Tree (AST) is built on a hierarchy where every specific expression (e.g., `BinaryExpr`, `LiteralExpr`) can be treated as a base `Expr` object. This allows the interpreter to process any node in the tree uniformly.
- **Dependency Inversion Principle (DIP):** High-level execution logic in the `Interpreter` depends on abstract `Expr` and `Stmt` classes rather than concrete implementations. This decoupling allows for the easy addition of new language features without modifying the core execution loop.

7 Tools and Technology

The project used the following tools:

- **Programming Language:** C++ (Standard 17) for manual control and performance.
- **Build System:** CMake for managing the project and making it work on different systems.
- **Version Control:** Git and GitHub for working together and tracking changes.
- **IDE:** CLion and Visual Studio Code for writing and debugging the code.
- **Diagrams:** Mermaid.js and Gamma for creating charts and architecture maps.

8 Application Domain

The project belongs to the domain of **Custom Programming Language Design**. It is built to learn about how an interpreter actually works and to provide a specialized tool for people who need quick calculations for physics and general unit conversions. For those who want to start programming without worrying about complex syntax and memory management, Drim offers a simple and user-friendly environment.

9 Project Timeline

The project was built in stages, starting with simple text reading and moving to complex features like functions and loops.

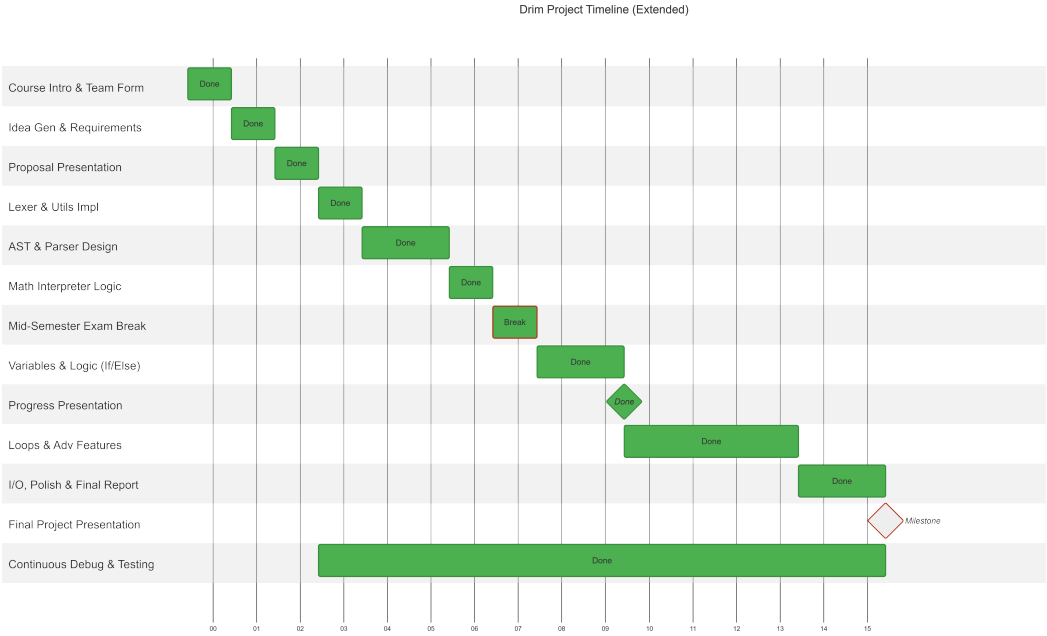

Figure 3: Project Development Timeline

10 Contribution

The work was divided among the team members as follows:

- **Md. Muntahi Hasan Akhiar:**
  - Created basic input and output systems and basic architecture of lexer, parser, and interpreter with Error Reporting.
  - Implemented variable management and automatic type casting.
  - Developed conditional logic and loops (`drimming`).
  - Added support for arrays and multi-assignment syntax.
  - Added modulo operations for math.
- **S.M. Tahsinuzzaman Emon:**
  - Implemented mathematical operations.
  - Built the physics function engine.
  - Added the Boolean data type.
  - Developed scoped variable logic.
  - Created the system for user-defined runtime functions.
  - Implemented String Interpolation and Output Formatting.
- **Tamim Mahrush Naser:**
  - Added unit conversion functions.
  - Worked on increasing internal calculation precision (int64).
  - Developed a Data Structures library (Stacks, Queues, etc.).

11 Future Work

We plan to keep improving Drim by adding more features. This includes adding many more functions for physics and unit conversions to make the language more useful. We also want to refine the core system and expand the built-in **Data Structures library** with more advanced types like Linked Lists and Trees. Other students have already started contributing on their forks, such as adding color highlighting for keywords, which we plan to include in the future.

12 Source Repository

The code is available at:
`https://github.com/hasanakhiar/drim-lang`